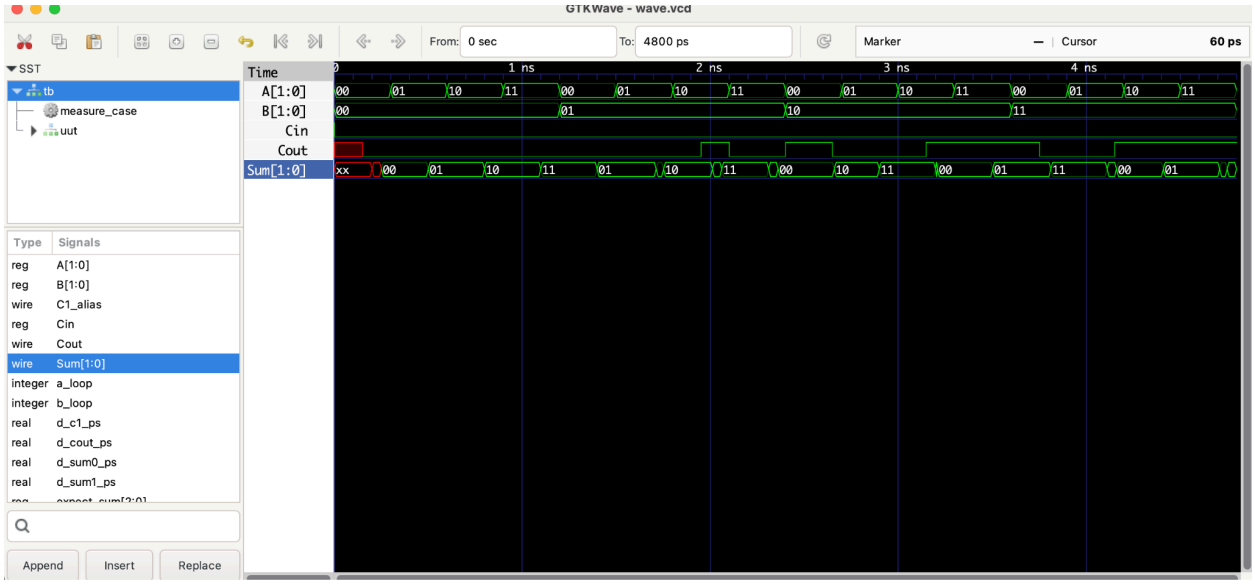
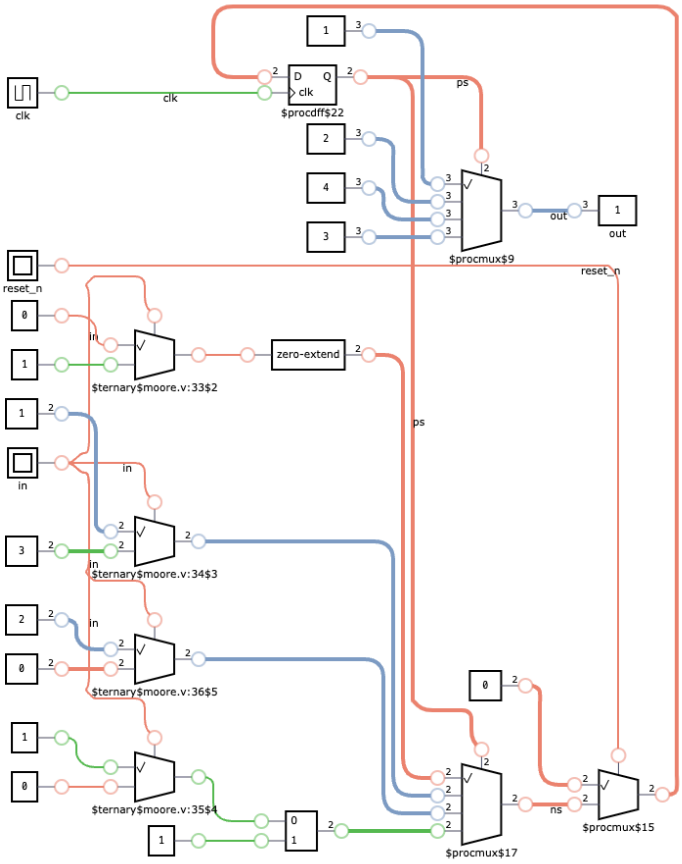
Olivia Chen
SID: 017098263
CMPE 124 Section 04

CMPE 124 LAB 5: Moore State Machines
PART 5a: Moore State Machine

Waveform Test for Specified Design Criteria:



Schematic (Generated via DigitalJS Online):



State Table:

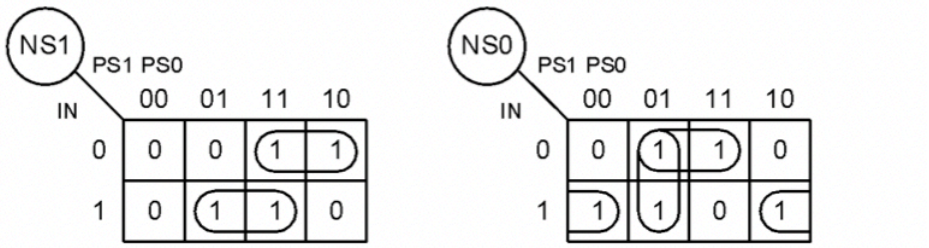
PS	NS		OUT
	IN = 0	IN = 1	
S0	S0	S1	1
S1	S1	S2	2
S2	S2	S3	3
S3	S3	S1	4

States	NS1	NS0
S0	0	0
S1	0	1
S2	1	1
S3	1	0

Transition Table:

PS1	PS0	IN = 0		IN = 1		OUT2	OUT1	OUT0
		NS1	NS0	NS1	NS0			
0	0	0	0	0	1	0	0	1
0	1	0	1	1	1	0	1	0
1	1	1	1	1	0	0	1	1
1	0	1	0	0	1	1	0	0

Logic:



$$NS1 = PS0.IN + PS1.\overline{IN}$$

$$NS0 = PS0.\overline{IN} + \overline{PS1}.PS0 + \overline{PS0}.IN$$

$$= (PS0 \oplus IN) + \overline{PS1}.PS0$$

$$OUT2 = PS1.\overline{PS0}$$

$$OUT1 = \overline{PS1}.PS0 + PS1.PS0 = PS0$$

$$OUT0 = \overline{PS1}.\overline{PS0} + PS1.PS0 = \overline{PS0 \oplus PS0}$$

Explanation: A Moore State Machine is a Finite State Machine (FSM), a digital circuit that moves between a limited number of states based on inputs and a clock. At each clock edge, the current/ present state is stored in flip flops, the next-state logic decides which state to go to next, and finally the output logic generates the circuit's outputs. A Moore-type machine's output

depends on the present state only, without considering inputs; this means that it is typically used in situations involving simpler, more stable output. Moore outputs change after a state change (low level-sensitive).
The Moore-type machine we are to implement in Part A is a 4-state Moore FSM with synchronous active-low reset and case-based transitions.The state table shown on the lab manual defines the states S0-S3 and the two variables: present state (ps) and next state (ns). The next state logic is: if IN=1, advance to the next state; if IN=0, stay put. The state register updates the present state on each rising clock edge, and when reset_n = 0, the FSM returns to the initial state (S0). Finally, the output logic maps each state to an output value, and depends only on ps, not on IN, matching the Moore definition.

Design Code:

```
`timescale 1ns/1ps
`default_nettype none

module moore(
   input wire clk,
   input wire reset_n,
   input wire in,
   output reg [2:0] out
);
//naming and defining the states
//enum logic = enumeration where each label responds to constant val
//enumeration = user-def data type, a set of named consants
// State encoding using parameters (plain Verilog)
parameter S0 = 2'b00,
          S1 = 2'b01,
          S2 = 2'b11,
          S3 = 2'b10;

reg [1:0] ps, ns;  // present state, next state

/*moore output depends ONLY on ps
BUT moore next state determined by input
(input doesn't directly influence output)
next-state logic computes ns based on ps & in
does NOT update state, only decides ns*/

//combinational logic block recomputes ns when in changes
always @* begin
   //defalut; assume ns stays same unless case overrides
   ns = ps;
   case (ps)
```

```
      //if ps=S0: in==1 -> ns=S1, in==0 -> ns=S0
      S0: ns = (in ? S1 : S0);
      S1: ns = (in ? S2 : S1);
      S2: ns = (in ? S3 : S2);
      S3: ns = (in ? S0 : S3);
      //if something illegal happens -> reset to S0
      default: ns = S0;
   endcase
end
// State register: synchronous reset to S0 when reset_n==0 (spec)
//rising edge triggers active low reset
always @(posedge clk) begin
 if (!reset_n)
     //nonblocking assignment
     ps <= S0;
 else
     ps <= ns;
end

// Moore output mapping (OUT = 1..4 per state)
//@* = "run block when ANY signal inside changes"
always @* begin
 //out depends on ps --> sets to 3 bit decimal
 case (ps)
   S0: out = 3'd1;
   S1: out = 3'd2;
   S2: out = 3'd3;
   S3: out = 3'd4;
   default: out = 3'd1;
 endcase
end

endmodule
```

Testbench Code:

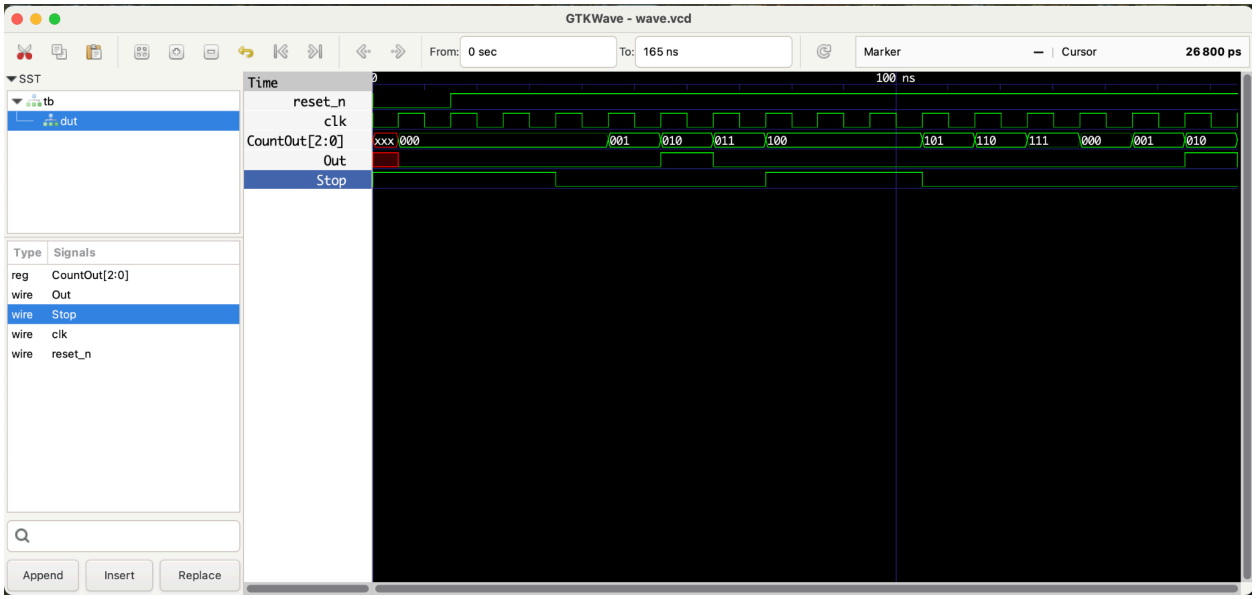
```
`timescale 1ns/1ps
`default_nettype none

module tb;
 reg clk = 0;
 reg reset_n = 0;
```

```
 reg in = 0;
 wire [2:0] out;


// DUT
moore dut (
 .clk(clk),
 .reset_n(reset_n),
 .in(in),
 .out(out)
);

 // 100 MHz clock (10 ns period)
 always #5 clk = ~clk;

 // Pretty-print helper
 task show(string tag);
   begin
     // Access internal state via hierarchical name: tb.dut.ps
     $display("[%0t] %s  ps=%b  out=%0d  in=%0b",
              $time, tag, tb.dut.ps, out, in);
   end
 endtask

 initial begin
 $dumpfile("wave.vcd");
 $dumpvars(0, tb);
   $display("=== Moore FSM smoke test ===");

   // Hold reset for 2 rising edges
   repeat (2) @(posedge clk);
   show("after reset low");
   reset_n = 1;
   @(posedge clk); show("released reset");

   // Hold (in=0): stay in S0, out should remain 1
   in = 0;
   repeat (3) @(posedge clk) show("hold (in=0)");

   // Advance (in=1): S0->S1->S2->S3->S0 ...
   in = 1;
   repeat (5) @(posedge clk) show("advance (in=1)");
```

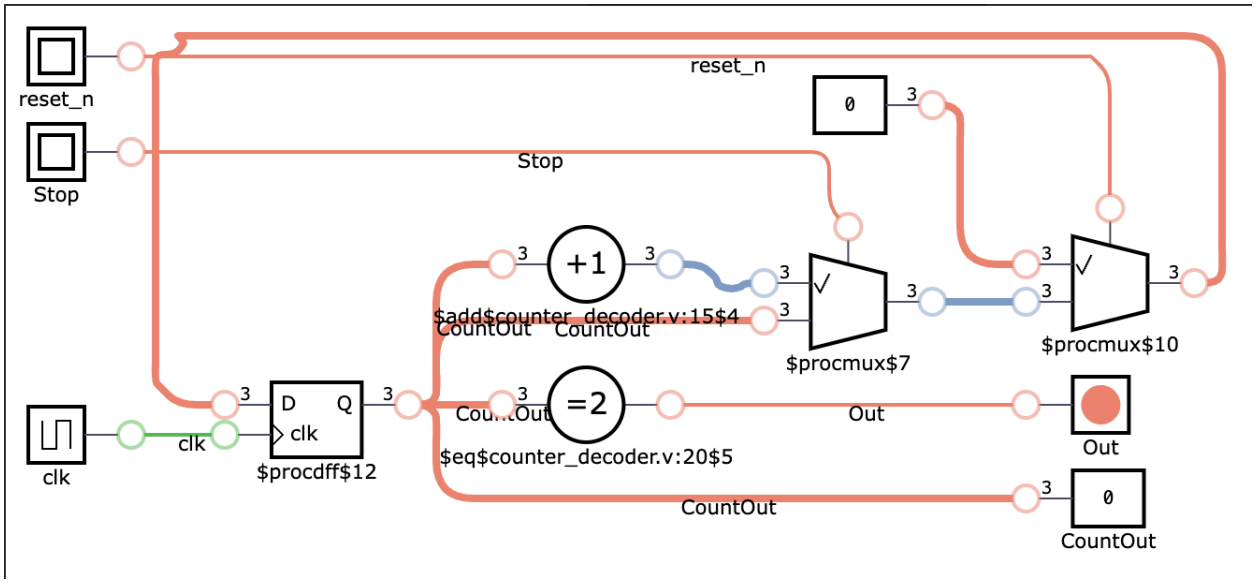
```
    // Mix hold/advance
    in = 0; @(posedge clk); show("hold");
    in = 1; @(posedge clk); show("advance");
    in = 0; @(posedge clk); show("hold");

    $display("=== Done ===");
    $finish;
  end
endmodule
```

PART 5b: Counter-Decoder Type Controllers

Waveform Test for Specified Design Criteria:

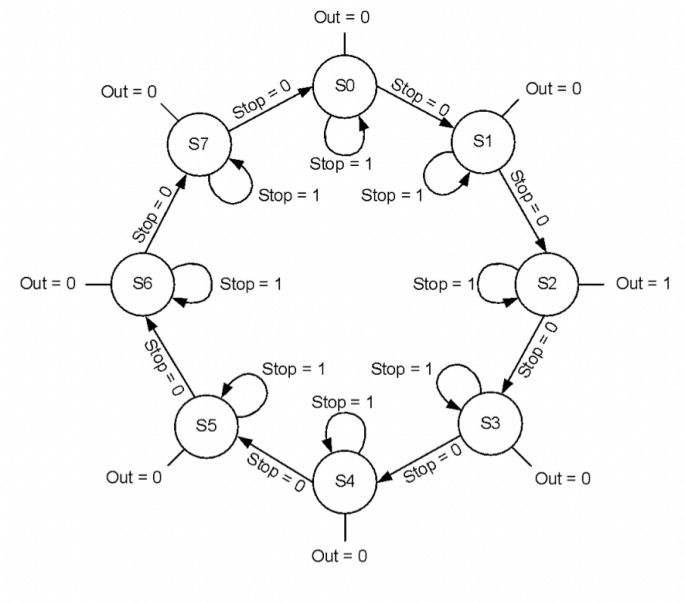


Schematic (Generated via DigitalJS Online):



Truth Table:

Logic:



Explanation: The Counter-Decoder type controller we are to implement in Part B is an 8-state Moore FSM where the next state is CountOut (present state)+1. There's no reset back to the previous state or the first state S0, and the Counter advances at Stop=0 (active low).

Design Code:

```
`timescale 1ns/1ps
`default_nettype wire

// counter_decoder.v
module counter_decoder (
 input  wire clk,
 input  wire reset_n,  // active-low, synchronous to clk
 input  wire Stop,     // 1 = hold, 0 = advance
 output reg  [2:0] CountOut, // present state 0-7
 output wire Out            // asserted only in state 2
);
 // 3-bit mod-8 counter with synchronous hold & reset
 always @(posedge clk) begin
   if (!reset_n) CountOut <= 3'd0;        // S0 on reset
   else if (!Stop) CountOut <= CountOut + 3'd1; // advance on Stop=0
   else           CountOut <= CountOut;   // hold on Stop=1
 end

 // One-hot-ish decoded output: Out high only in S2
 assign Out = (CountOut == 3'd2);
endmodule
```

Testbench Code:

```
// tb_counter_decoder.v
`timescale 1ns/1ps
module tb;
 reg clk=0, reset_n=0, Stop=1;
 wire [2:0] CountOut;
 wire Out;
 counter_decoder dut(.clk(clk), .reset_n(reset_n), .Stop(Stop), .CountOut(CountOut),
.Out(Out));
 always #5 clk = ~clk;

 initial begin
   $dumpfile("wave.vcd");
   $dumpvars(0, tb);

   // reset for two cycles
   repeat (2) @(posedge clk);
   reset_n = 1;

   // Hold at S0 (Stop=1)
   repeat (2) @(posedge clk);

   // Walk S0->S1->S2->S3 with Stop=0
   Stop = 0; repeat (4) @(posedge clk);

   // Hold in S4
   Stop = 1; repeat (3) @(posedge clk);

   // Advance through wrap 5..7->0->1->2 (Out should pulse at S2)
   Stop = 0; repeat (6) @(posedge clk);

   $finish;
 end
endmodule
```